

time the function is called instead of *just once* when the function is defined, such as in the following example:

```
>>> def addIngredient(ingredient, sandwich=None):
...     if sandwich is None:
...         sandwich = ['bread', 'bread']
...     sandwich.insert(1, ingredient)
...     return sandwich
...
>>> firstSandwich = addIngredient('cranberries')
>>> firstSandwich
['bread', 'cranberries', 'bread']
>>> secondSandwich = addIngredient('lettuce')
>>> secondSandwich
['bread', 'lettuce', 'bread']
>>> id(firstSandwich) == id(secondSandwich)
❶ False
```

Notice that `firstSandwich` and `secondSandwich` don't share the same list reference ❶ because `sandwich = ['bread', 'bread']` creates a new list object each time `addIngredient()` is called, not just once when `addIngredient()` is defined.

Mutable data types include lists, dictionaries, sets, and objects made from the `class` statement. *Don't* put objects of these types as default arguments in a `def` statement.

Don't Build Strings with String Concatenation

In Python, strings are *immutable* objects. This means that string values can't change, and any code that seems to modify the string is actually creating a new string object. For example, each of the following operations changes the content of the `spam` variable, not by changing the string value, but by replacing it with a new string value that has a new identity:

```
>>> spam = 'Hello'
>>> id(spam), spam
(38330864, 'Hello')
>>> spam = spam + ' world!'
```